

Facade

1. Explicación del patrón

Facade es un patrón estructural que proporciona una interfaz simple y unificada para acceder a un conjunto de clases, objetos o subsistemas complejos.

La idea principal es crear una clase intermedia, llamada fachada, que actúa como punto de entrada al sistema.

En lugar de que el cliente tenga que conocer y utilizar directamente muchas clases internas, el cliente se comunica únicamente con la fachada.

La fachada se encarga internamente de llamar a las clases necesarias del subsistema, coordinar sus métodos y adaptar las operaciones para ofrecer una interfaz más sencilla.

De esta forma, el sistema sigue manteniendo sus clases internas y su modularidad, pero el cliente no queda acoplado a todos esos detalles.

2. Cuándo se utiliza

Se utiliza el patrón Facade cuando:

- Se quiere proporcionar una interfaz simple para un sistema complejo.
- Un subsistema tiene muchas clases pequeñas que el cliente no necesita conocer directamente.
- Se quiere facilitar el uso de un conjunto de clases.
- Se quiere reducir el acoplamiento entre el cliente y las clases internas del sistema.
- Se quiere dividir un sistema en subsistemas.
- Se quiere que la comunicación entre subsistemas se realice de forma más abstracta.
- Se quiere encapsular la implementación interna de un sistema de cara al exterior.
- Se quiere evitar que el cliente tenga que saber el orden exacto en el que debe llamar a varios métodos.
- Se quiere centralizar la lógica de acceso a un subsistema.
- Se quiere proteger al cliente de cambios internos en las clases del subsistema.

3. Cómo aplicarlo

Para aplicar Facade se suelen definir estos participantes:

Fachada

Es la clase principal del patrón.

Responsabilidades:

- Exponer una interfaz sencilla al cliente.
- Mantener referencias a las clases o componentes del subsistema.
- Coordinar las llamadas internas necesarias.

- Adaptar la interfaz de bajo nivel del subsistema a una interfaz de nivel más alto.
- Ocultar la complejidad interna al cliente.

La fachada no debe limitarse siempre a reenviar llamadas.

En muchos casos, también contiene código específico para combinar métodos internos, preparar datos o llamar a varias clases en el orden correcto.

Ejemplo conceptual:

- Facade
- Fachada
- MultimediaFacade

Subsistema

Es el conjunto de clases que implementan la funcionalidad real.

Responsabilidades:

- Realizar las operaciones concretas del sistema.
- Responder a las llamadas de la fachada.
- Trabajar entre sí si es necesario.
- No depender de la fachada para funcionar.

Las clases del subsistema no necesitan conocer que existe una fachada.

Ejemplos conceptuales:

- Clase1
- Clase2
- Clase3

Ejemplos del material:

- Compra
- Inventario
- Pago
- Envio
- AudioPlayer
- VideoPlayer
- Lights

Cliente

Es quien necesita usar el sistema.

Responsabilidades:

- Comunicarse con la fachada.

- Evitar depender directamente de todas las clases internas.
- Usar una interfaz más simple y de mayor nivel.

El cliente puede seguir accediendo directamente al subsistema si lo necesita, pero el objetivo del patrón es que use la fachada para las operaciones habituales.

4. Forma práctica de aplicarlo

Pasos:

1. Identificar un conjunto de clases que el cliente está usando directamente.
2. Comprobar si el cliente necesita conocer demasiados detalles internos.
3. Detectar las operaciones más comunes que el cliente necesita realizar.
4. Crear una clase fachada.
5. Añadir en la fachada referencias a las clases del subsistema.
6. Crear métodos sencillos en la fachada que representen acciones de alto nivel.
7. Dentro de cada método de la fachada, llamar a las clases internas necesarias.
8. Coordinar el orden de las llamadas si es importante.
9. Adaptar datos o resultados si el cliente necesita una respuesta más simple.
10. Modificar el cliente para que use la fachada en lugar de acceder directamente al subsistema.

5. Idea clave

El cliente no debe tener que conocer toda la complejidad interna del subsistema.

La fachada ofrece un punto de entrada sencillo.

Así, el cliente realiza una operación de alto nivel y la fachada se encarga de coordinar las clases internas necesarias.

6. Funcionamiento interno

El funcionamiento habitual es:

1. El cliente llama a un método de la fachada.
2. La fachada recibe la petición.
3. La fachada ejecuta código propio si necesita preparar la operación.
4. La fachada llama a una o varias clases del subsistema.
5. Las clases internas realizan el trabajo concreto.
6. La fachada puede combinar o adaptar los resultados.
7. El cliente recibe una respuesta simple.

Ejemplo conceptual:

El cliente llama a:

```
fachada.solicita()
```

La fachada internamente puede hacer:

- `calcula1()`
- `claseB1.solicita()`
- `calcula2()`

El cliente no necesita conocer esos pasos internos.

7. Ejemplos del material

Ejemplo general

En el diagrama general del material aparece una clase `Fachada` con métodos como:

- `solicita1()`
- `solicita2()`
- `solicita3()`
- `calcula1()`
- `calcula2()`

La fachada se comunica con distintas clases y componentes del sistema.

El cliente llama a la fachada, y la fachada decide qué clases internas deben intervenir.

La estructura general sería:

`Cliente -> Fachada -> Subsistema`

Ejemplo de compra online

En el material aparece un ejemplo de compra online.

Sin fachada, el cliente tendría que comunicarse directamente con varias clases:

- `Compra`
- `Inventario`
- `Pago`
- `Envio`

Cada una tiene su propia responsabilidad:

- `Compra` : realizar la compra.
- `Inventario` : verificar el stock.
- `Pago` : procesar el pago.
- `Envio` : enviar el producto.

El problema es que el cliente queda demasiado ligado a todos esos pasos.

Con Facade, se introduce una clase `Fachada` .

Entonces el cliente solo se comunica con la fachada, y la fachada coordina internamente las operaciones de compra, inventario, pago y envío.

Así, el cliente no necesita saber cómo se organiza internamente el proceso de compra online.

Ejemplo de equipo multimedia

El material también muestra un ejemplo de equipo multimedia.

Aparecen las clases internas:

- `AudioPlayer`
- `VideoPlayer`
- `Lights`

Con métodos como:

- `play()`
- `turnON()`
- `turnOFF()`

La fachada sería:

- `MultimediaFacade`

El cliente se comunica con `MultimediaFacade` en lugar de manipular directamente cada componente.

La fachada puede encargarse de encender luces, reproducir audio y reproducir vídeo mediante una operación más simple.

8. Ejemplo conceptual de uso

Sin Facade, el cliente podría tener que hacer algo como:

```
inventario.verificarStock()
```

```
pago.procesarPago()
```

```
compra.realizarCompra()
```

```
envio.enviarProducto()
```

Con Facade, el cliente podría hacer simplemente:

```
fachada.realizarCompraOnline()
```

La fachada se encarga de ejecutar todos los pasos internos necesarios.

9. Ventajas prácticas

- Simplifica el uso de sistemas complejos.
- Reduce el acoplamiento entre cliente y subsistema.
- Oculta detalles internos de implementación.
- Centraliza la forma de acceder al subsistema.
- Facilita modificar el subsistema sin afectar tanto al cliente.
- Permite dividir un sistema en capas o subsistemas.
- Hace que el código cliente sea más claro.

- Evita que el cliente tenga que coordinar muchas clases manualmente.
- Permite ofrecer solo las operaciones que realmente necesita el cliente.

10. Cuidado

Facade puede convertirse en una clase demasiado grande si se le añaden demasiadas responsabilidades.

Si la fachada empieza a incluir muchas operaciones no relacionadas, puede acabar siendo una clase difícil de mantener.

En ese caso, puede ser mejor crear varias fachadas, cada una para una parte concreta del sistema.

También hay que recordar que Facade no elimina la complejidad del subsistema.

Solo la oculta y la organiza detrás de una interfaz más simple.

11. Diferencia práctica con otros patrones estructurales

Diferencia con Adapter

Adapter convierte una interfaz incompatible en otra que el cliente espera.

Facade proporciona una interfaz más simple para un subsistema completo.

Por tanto:

- Adapter se usa cuando las interfaces no encajan.
- Facade se usa cuando el sistema es complejo y se quiere simplificar su uso.

Diferencia con Decorator

Decorator añade funcionalidades a un objeto manteniendo su interfaz.

Facade no añade funcionalidades al subsistema, sino que simplifica el acceso a él.

Por tanto:

- Decorator extiende comportamiento.
- Facade simplifica el uso.

Diferencia con Composite

Composite organiza objetos en forma de árbol y permite tratar hojas y compuestos igual.

Facade no representa una jerarquía, sino que agrupa el acceso a varias clases internas.

Por tanto:

- Composite se centra en estructuras parte-todo.
- Facade se centra en ofrecer una entrada simple a un subsistema.

Diferencia con Proxy

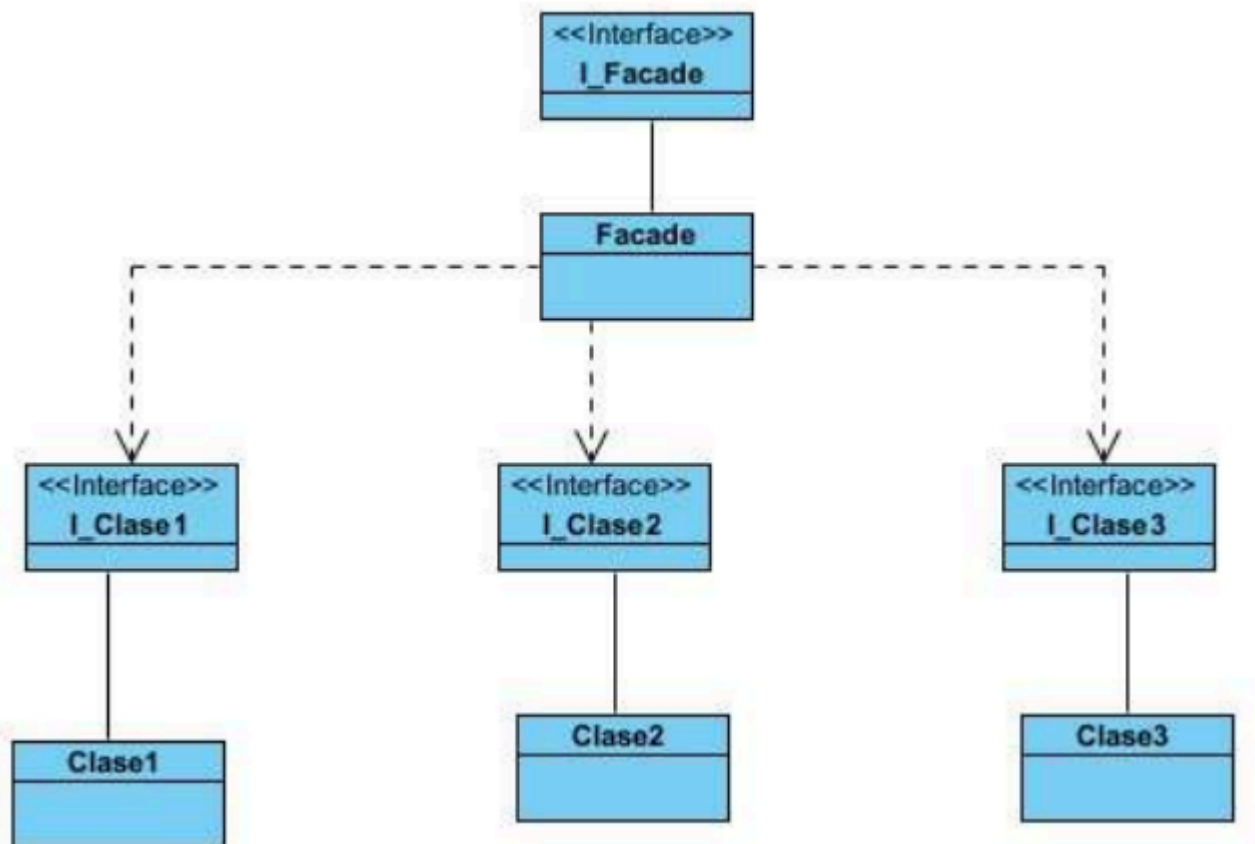
Proxy controla el acceso a un objeto concreto.

Facade simplifica el acceso a un conjunto de objetos.

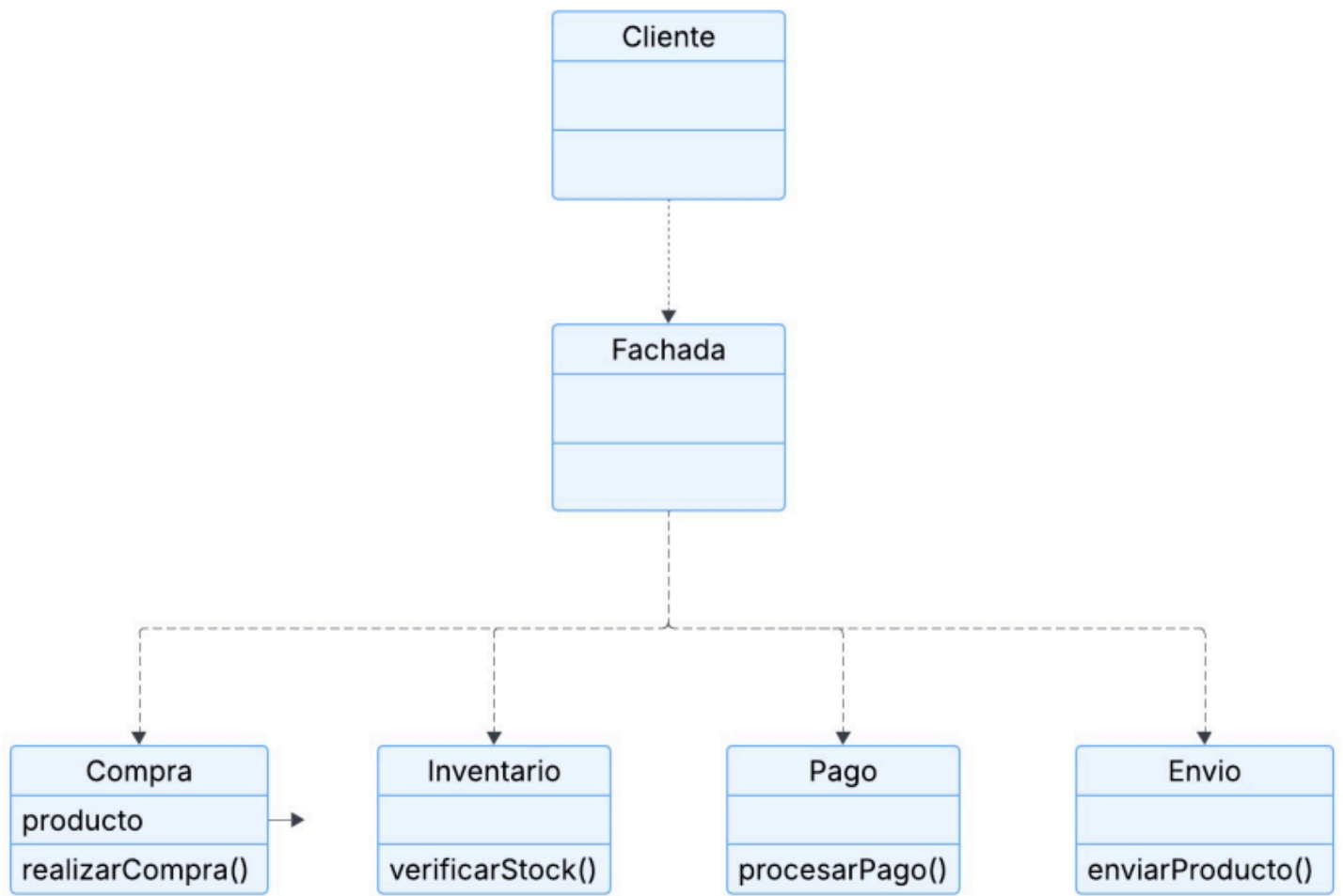
Por tanto:

- Proxy actúa como intermediario de un objeto.
- Facade actúa como punto de entrada a un subsistema.

12. Plantilla



13. Ejemplo



14. Ejemplo2

